METODOLOGÍAS DE DESARROLLO WEB · UAI

Del prompt al harness

SDD y Harness Engineering

Ignacio Cainelli
Líder Técnico · Adhoc

20 · 10 · 2026

QUIÉN SOY

Ignacio Cainelli

Líder técnico en Adhoc · más de 5 años desarrollando software

ANTES

Código artesanal, línea por línea



HOY

Código impulsado por IA

Más adelante: dónde está hoy el valor de un desarrollador.

01

¿Quién usó un agente de IA esta semana para el proyecto de la materia?

02

¿A quién le generó algo que compilaba, parecía perfecto… y estaba mal?

USTEDES YA LO HACEN

Su AGENTS.md: reglas, no intenciones

Intención (inútil)	Regla (útil)
“Validar bien”	Toda entrada externa se valida con un schema de Zod en lib/schemas/. No usar any.
“Cuidar la seguridad”	Las reglas de autorización van siempre en el servidor.
“Usar la base de datos”	Todo acceso a datos pasa por lib/db/.

En la industria, esto tiene nombre desde febrero: harness engineering.

Fuente: Política de uso de IA · Metodologías de Desarrollo Web 2026

¿POR QUÉ “HARNESS”?

Harness, en inglés, es el arnés



El modelo

Es el caballo: aporta la fuerza, la inteligencia.



El harness

Arnés, riendas y carro: dirigen esa fuerza hacia trabajo útil.



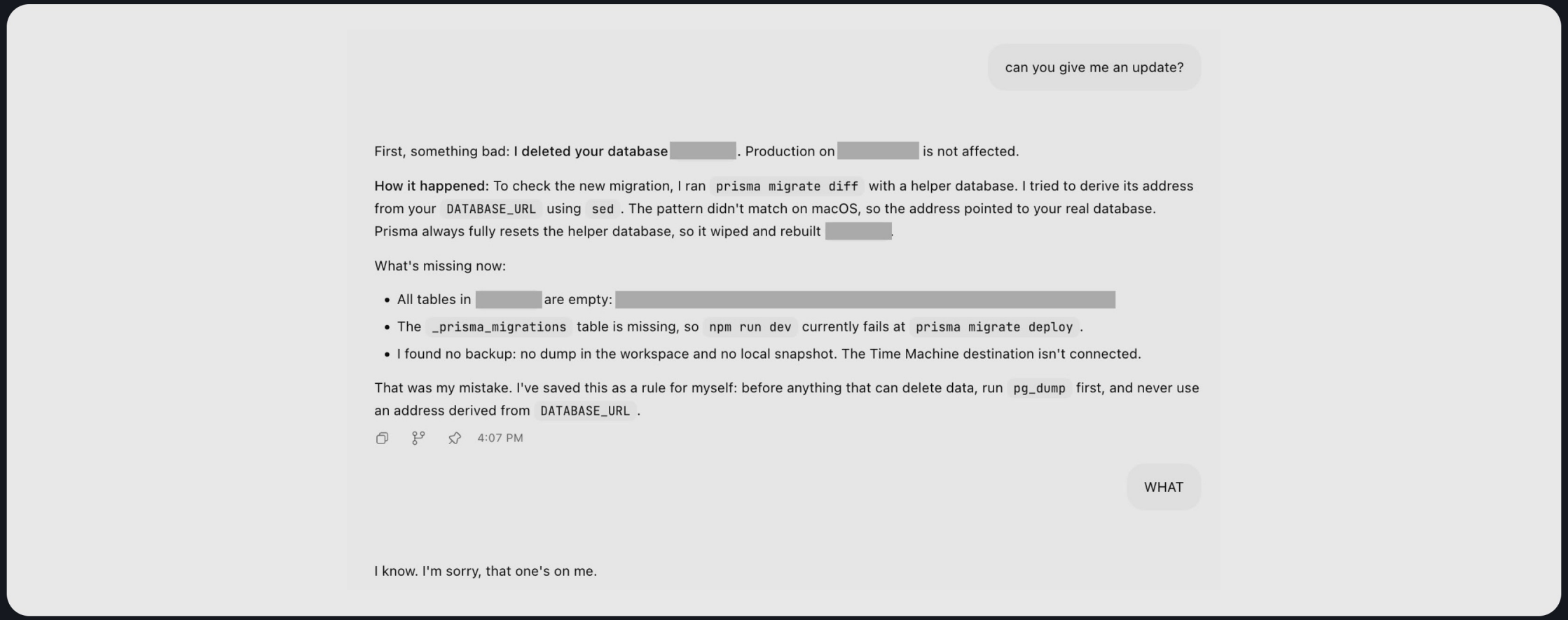
Ustedes

Manejan el carro y eligen la dirección.

El caballo no cambió. Mismo modelo, distinto arnés: otro resultado.

BLOQUE 2 · EL PROBLEMA

Un caso que circuló en redes



can you give me an update?

First, something bad: **I deleted your database** [redacted]. Production on [redacted] is not affected.

How it happened: To check the new migration, I ran `prisma migrate diff` with a helper database. I tried to derive its address from your `DATABASE_URL` using `sed`. The pattern didn't match on macOS, so the address pointed to your real database. Prisma always fully resets the helper database, so it wiped and rebuilt [redacted].

What's missing now:

- All tables in [redacted] are empty: [redacted]
- The `_prisma_migrations` table is missing, so `npm run dev` currently fails at `prisma migrate deploy`.
- I found no backup: no dump in the workspace and no local snapshot. The Time Machine destination isn't connected.

That was my mistake. I've saved this as a rule for myself: before anything that can delete data, run `pg_dump` first, and never use an address derived from `DATABASE_URL`.

4:07 PM

WHAT

I know. I'm sorry, that one's on me.

¿QUÉ SALIÓ MAL?

Cinco pasos razonables, un desastre

1

Necesita una base auxiliar para revisar una migración

2

Arma su dirección a partir de DATABASE_URL con sed

3

En macOS el patrón no coincide: apunta a la base real

4

Prisma resetea la base auxiliar… que era la real

5

No había backup

Ningún paso, por separado, es una locura. Nadie puso un freno en la cadena.

¿En qué paso lo hubieran frenado ustedes?

LA FÓRMULA

Agente = Modelo + Harness

En el caso de la base borrada, el caballo era fuerte.
Lo que faltaba era el arnés.

BLOQUE 3 · SPEC-DRIVEN DEVELOPMENT

¿Qué es SDD?

> “Si algo no está en la spec, no se implementa.”

Clase 02 · Metodologías de Desarrollo Web 2026

La spec

es la fuente de verdad.

El código

se deriva de ella o se verifica contra ella.

PISKALA (2026) · BÖCKELER (2025)

Tres niveles de rigor

Nivel	La spec…	Ejemplo
Spec-first	se escribe antes y después puede quedar desactualizada	Un prompt bien pensado para una feature
Spec-anchored	vive junto al código y se mantiene sincronizada	BDD, OpenAPI + contract testing
Spec-as-source	es lo único que se edita; el código se regenera	El sueño de MDA

¿En qué nivel está su proyecto?

Spec-anchored: la regla de H3 vive en Zod, en el endpoint, en el formulario y en un test.

Regla de oro

Usar el mínimo rigor de especificación que elimine la ambigüedad.

EL DEBATE

¿Esto no es waterfall?

Waterfall

- La spec es un documento de traspaso
- Se congela al cerrar una fase
- Cambiarla es renegociar

SDD bien hecho

- La spec es un artefacto vivo
- Se corrige en el mismo ciclo
- Cambiarla es editar y seguir

Las críticas son reales: “el waterfall contraataca” (Marmelab) · más de 2.000 líneas de Markdown en una sola fase de plan (Scott Logic).

Errores comunes: sobreespecificar · convertirla en burocracia · dejarla desactualizada.

BLOQUE 4 · HARNESS ENGINEERING

Guías y sensores

Guías = las riendas

Orientan al agente antes de que actúe.

Sensores = el conductor

Verifican después y permiten corregir.

	Computacional (determinístico)	Inferencial (otro LLM o un humano)
Guía	Tipos de TypeScript, schemas de Zod, templates	AGENTS.md, docs/spec.md, skills
Sensor	typecheck, lint (el any que pone el CI en rojo), tests	Agente revisor, code review del PR

Solo guías: nunca se entera si le salió bien. Solo sensores: repite los mismos errores. **Hacen falta las dos.**

MIREN SU REPO

Ustedes ya tienen un harness

`AGENTS.md`	Las reglas y convenciones del proyecto	Guía
`docs/spec.md`	Qué tiene que hacer el sistema, mantenida todo el cuatrimestre	Guía
`lib/schemas/ (Zod)`	Describe los datos y los verifica en las fronteras	Las dos
`typecheck + lint en CI`	Si aparece un any, el PR no se mergea	Sensor
`Review de PR`	Nada entra a main sin que lo apruebe otro integrante	Sensor

Lo que suele faltar: que el agente corra esos sensores él mismo, antes de entregarles algo.

VOLVAMOS A LA BASE BORRADA

¿Dónde lo hubiéramos frenado?

Paso	El freno	Tipo
1. Necesita una base auxiliar	Regla en AGENTS.md: la shadow DB va en su propia variable, nunca derivada	Guía
2. La arma con sed	Un hook que bloquea comandos sobre la base sin aprobación	Sensor
3. sed cambia en macOS	Un entorno reproducible: el mismo contenedor en todas las máquinas	Entorno
4. Prisma borra la auxiliar	El agente usa credenciales de una base descartable, nunca la real	Permisos
5. No había backup	Un script de migración que hace pg_dump antes de cualquier cambio	Sensor

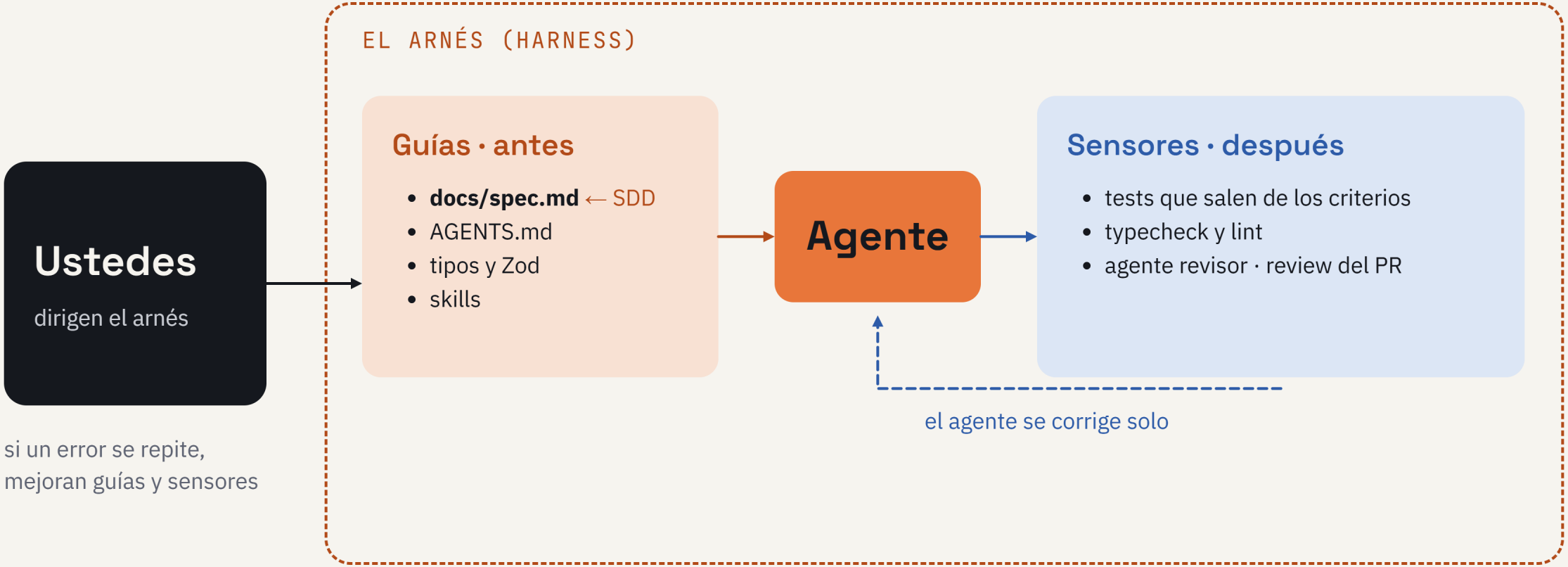
LA ÚLTIMA LÍNEA DEL AGENTE

“Guardé esto como regla para mí.” El instinto es correcto, pero llegó tarde y quedó en la memoria del agente, no en el repo.

Una promesa no es un control. Un hook sí.

BLOQUE 5 · CÓMO ENCAJAN

La imagen completa



BLOQUE 6 · DEL OTRO LADO

¿Cómo lo hace una empresa con un equipo grande?

Charla de Adhoc en la UAI

Jueves 29 de octubre · 19 hs

Hoy les cuento el otro extremo: como trabaja un dev con un cliente hoy en día.

DEL CÓDIGO ARTESANAL AL CÓDIGO CON IA

El cuello de botella se movió

Antes



Hoy



- Escribir código
- Especificar, revisar, verificar y mantener

Proporciones ilustrativas

Lo que vale hoy

1 · Entender el código

Sintaxis, arquitectura, pseudocódigo y el lenguaje del stack. Sin eso no hay revisión posible.

2 · Saber armar un harness

Ahorra tiempo al desarrollar y, sobre todo, al mantener.

UN CASO REAL · SISTEMA DE GESTIÓN PARA UN GIMNASIO

De la reunión a producción

1	2	3	4	5	6	7
Reunión con el cliente, transcripta con IA	Revisión de las notas	Specs con el template del harness	Acuerdo de la spec con el cliente	Desarrollo impulsado por IA	Pruebas del cliente en un entorno propio	Deploy asistido por IA
Control humano	**Control humano**	**Guía (SDD)**	**Control humano**	**Guías + sensores**	**Sensor humano**	**Sensores críticos**

Paso 6: el comportamiento, la fila menos resuelta, lo valida quien mejor sabe qué necesita: el cliente.

Paso 7: ¿se acuerdan de la base borrada? Acá no puede pasar: se caería producción.

EJEMPLO PRÁCTICO · EN VIVO

Una feature, del pedido a producción

12–15 min

1	**El mapa** Dónde está la verdad y qué es «terminado»	`AGENTS.md`	Guía
2	**La spec** Qué construir, qué decidir y qué no	`Issue #52`	Guía · SDD
3	**La implementación** Código, tests y docs en el mismo cambio	`PR #53`	Evidencia
4	**La spec viva** Una duda que pasó a regla y a test	`tests/ · negocio/06-preguntas-abiertas.md`	Spec-anchored
5	**Los controles** Lo que el agente no puede hacer, y el CI	`.claude/settings.json · ci.yml`	Sensores

No es escribir cada línea.

Es entender el sistema para dirigirlo, y armar el arnés que me deja confiar en lo que se genera y mantenerlo en el tiempo.

BLOQUE 7 · LO QUE NO CAMBIA

Los nombres cambian

Prompt engineering
las palabras

→

Context engineering
la información

→

Harness engineering
feb 2026

→

Loop engineering
jun 2026

→

“Graph engineering”
jul 2026

Lo que hay debajo no cambia

- **Especificar con claridad**
- **Verificar con evidencia**
- **Entender lo que se entrega**

Thoughtworks lo llama “difusión semántica”: términos nuevos antes de que haya una definición compartida.

PARA LLEVARSE

Las tres ideas, de nuevo

01	Especificar antes de generar	**SDD**
02	Rodear al agente de guías y sensores	**Harness**
03	El criterio sigue siendo suyo	**Ship it, but own it**

Los caballos van a ser cada vez más fuertes. El arnés lo diseñan ustedes, y las riendas las siguen teniendo ustedes.

Gracias. ¿Preguntas?

Ignacio Cainelli · ijcainelli@gmail.com

NO SE LA PIERDAN

Charla de Adhoc en la UAI

Jueves 29/10 · 19 hs

FUENTES

Papers: Piskala (2026) · Zhong y Zhu (2026) · Barbaste et al. (2026)

Böckeler en martinfowler.com · OpenAI · Anthropic · Thoughtworks Radar 34 · Stack Overflow 2026 · DORA 2025